

Constrained Discrete Diffusion Models for Graph Generation

Nico Ohler

Seminar Project

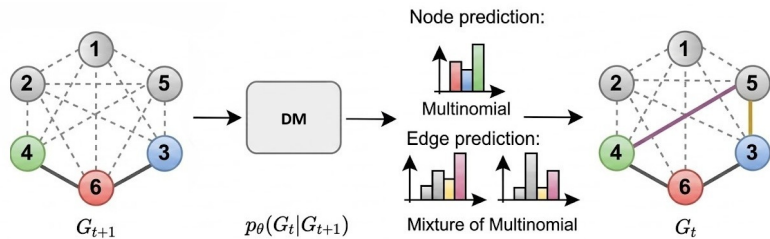
Welcome everyone, to the presentation of my seminar project **"Constrained Discrete Diffusion Models for Graph Generations"**.



Introduction

Traditional discrete graph diffusion models

- predict transition probabilities for all node and edge attributes
- restricted to small graphs due to quadratic space complexity



Kong et al., *Autoregressive Diffusion Model for Graph Generation*

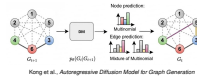
Constrained Graph Diffusion

Introduction

Introduction

Introduction

- Traditional discrete graph diffusion models
 - predict transition probabilities for all node and edge attributes
 - restricted to small graphs due to quadratic space complexity



Kong et al., *Autoregressive Diffusion Model for Graph Generation*

Recently, discrete graph diffusion models have seen a lot of success in various domains such as drug discovery and 3D modeling. **Unlike their continuous counterparts, they receive** a noisy graph as input and predict state transition **probabilities for node and edge attributes** - for example node classes or edge weights. **The predicted distribution will** then be used to sample a completely new graph for next time step. This **process is repeated** for a fixed number of times until you arrive at **final output graph**. **While this approach works quite well in practice, it is unfortunately limited to relatively small graphs since space complexity grows quadratically** with number of nodes.

Introduction

- Traditional discrete graph diffusion models
 - predict transition probabilities for all node and edge attributes
 - restricted to small graphs due to quadratic space complexity
- Efficient sampling of sparse graphs via SparseDiff
- Evaluate constraint satisfaction capabilities
- Reduce constraint violations via
 - Projected Diffusion
 - Unlikelihood Training

Constrained Graph Diffusion

└ Introduction

└ Introduction

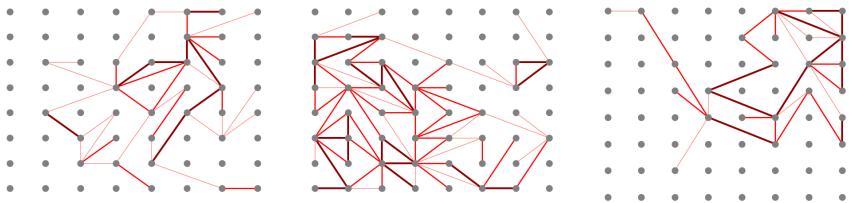
This is the main motivation as to why we decided to test a relatively new approach called SparseDiff, **which allows efficient sampling of sparse graphs. And since real world** applications demand not only similar looking graphs but also plausible ones, **we also evaluated whether the learned** model is able to generate new graphs that adhere to **certain constraints, that are present** in the training data. **Afterwards, we tried to reduce** the number of constraint violations via projected diffusion and unlikelihood training **two methods which** I will later explain in more detail.

Introduction

- Traditional discrete graph diffusion models
 - predict transition probabilities for all node and edge attributes
 - restricted to small graphs due to quadratic space complexity
- Efficient sampling of sparse graphs via SparseDiff
- Evaluate constraint satisfaction capabilities
- Reduce constraint violations via
 - Projected Diffusion
 - Unlikelihood Training

Synthetic Dataset

- 5000 undirected graphs on a 2D 8x8 coordinate grid
- Each graph satisfies these constraints
 - planarity
 - connectivity
 - edge lengths $l \in \{1, \sqrt{2}, \sqrt{5}\}$



Sample graphs from the synthetic dataset

2026-07-22

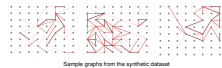
Constrained Graph Diffusion

└ Methodology

└ Synthetic Dataset

Synthetic Dataset

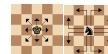
- 5000 undirected graphs on a 2D 8x8 coordinate grid
- Each graph satisfies these constraints
 - planarity
 - connectivity
 - edge lengths $l \in \{1, \sqrt{2}, \sqrt{5}\}$



Sample graphs from the synthetic dataset

For the experiment we created a synthetic dataset consisting of 5000 undirected graphs on a 2D 8x8 coordinate grid **where each graphs satisfies three constraints.** planarity, meaning there are no edges that cross each other
 connectivity, meaning that **each node is reachable from every other node by traversing existing edges**
 and we also limited edges to a lengths of either ...

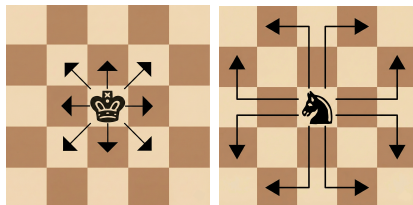
- 5000 undirected graphs on a 2D 8x8 coordinate grid
- Each graph satisfies these constraints
 - planarity
 - connectivity
 - edge lengths $l \in \{1, \sqrt{2}, \sqrt{5}\}$



©Encyclopedia Britannica, Inc.

Synthetic Dataset

- 5000 undirected graphs on a 2D 8x8 coordinate grid
- Each graph satisfies these constraints
 - planarity
 - connectivity
 - edge lengths $l \in \{1, \sqrt{2}, \sqrt{5}\}$

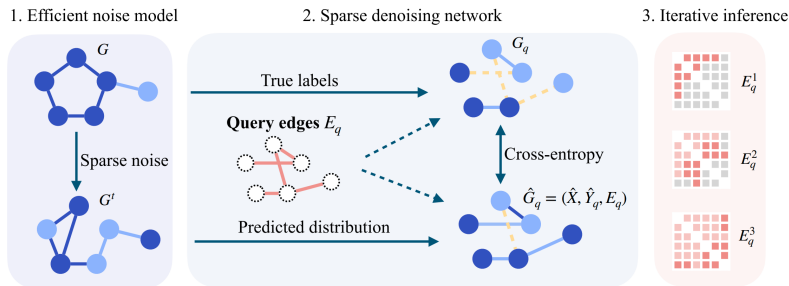


©Encyclopedia Britannica, Inc.

For those familiar with chess, you can intuitively think of these edge lengths as the union of king and knight moves.

SparseDiff

- Leverages inherent sparsity of most large graphs
- Provides control over space complexity



Qin, Vignac, and Frossard, "SparseDiff: Sparse Discrete Diffusion for Scalable Graph Generation"

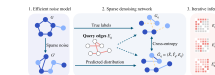
Constrained Graph Diffusion

└ Methodology

└ SparseDiff

SparseDiff

- Leverages inherent sparsity of most large graphs
- Provides control over space complexity



Qin, Vignac, and Frossard, "SparseDiff: Sparse Discrete Diffusion for Scalable Graph Generation"

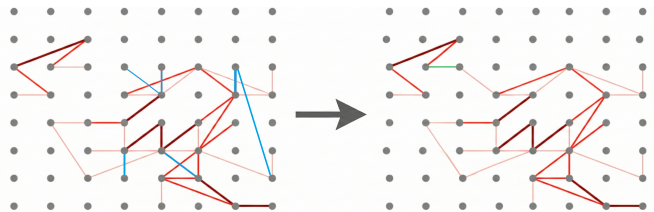
For graph generation we used SparseDiff, which unlike traditional approaches does not need to compute dense. It instead leverages inherent sparsity via 3 components:

1 efficient noise model allows efficient generation of noisy graphs at specific timesteps based on real graphs from dataset and will be used to create **noisy training samples on the fly**
2 During training, these noisy graphs serve as input to the sparse denoising network which learns to predict the transition probabilities for all nodes and a subset of all edges by minimizing the Cross Entropy Loss between the predicted distribution and the true, clean graph

3 During the iterative inference, the denoising model is then used to **iteratively predict and sample** from transition probabilities, **first for first** subset of all edges, then second subset, **until you have covered all possible edges** and sampled a new

Projected Diffusion

- Projecting intermediate outputs of every diffusion step to closest feasible point
- Correcting graphs to satisfy all constraints
 - remove all edges with illegal lengths
 - detect and resolve edge crossings
 - connect disconnected subgraphs



Constrained Graph Diffusion

└ Methodology

└ Projected Diffusion

Projected Diffusion

- Projecting intermediate outputs of every diffusion step to closest feasible point
- Correcting graphs to satisfy all constraints
 - remove all edges with illegal lengths
 - detect and resolve edge crossings
 - connect disconnected subgraphs

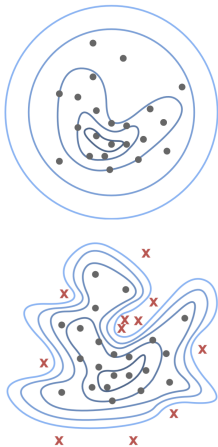


Next, we will talk about the concept of projected diffusion, which is **defined as projecting the output of every** Or in other words, we correct predicted graphs to ... with as **few changes as possible**. For our project, we ... In this example, the **disconnected graph on the left contains one edge that is too long** - this one - and several edge crossings. **projected version on the right** removed the **edges highlighted in blue** and **added a single edge** - highlighted in green - to connect the graph.

While this approach guarantees constraint satisfaction, it also comes with... **The projection may be difficult** to implement, slow inference and potentially lead to distribution shifts.

Unlikelihood Training

- Explicitly penalizing undesired outputs
- Fine-tune an existing model by penalizing predictions that create edge crossings
- Compute cross-entropy loss between predicted and true distribution for some edges that would cross existing edges
- Learn a tighter distribution



Regenwetter et al., "Constraining Generative Models for Engineering Design with Negative Data"

2026-07-22

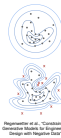
Constrained Graph Diffusion

└ Methodology

└ Unlikelihood Training

Unlikelihood Training

- Explicitly penalizing undesired outputs
- Fine-tune an existing model by penalizing predictions that create edge crossings
- Compute cross-entropy loss between predicted and true distribution for some edges that would cross existing edges
- Learn a tighter distribution



Because of that, we also tested another method called unlikelihood training to see whether it is a viable alternative. defined as explicitly penalizing, **thus less likely**.

In this project, we used it to fine-tune an existing model by **penalizing edge crossings, since the planarity constraint** turned out.

We do this by computing the cross-entropy loss between the predicted and the true distribution for **certain edges, that would cross existing ones from the noisy ground truth**.

By doing so, we hope to also gain a **tighter, more accurate fit** to the **true data distribution**.

Training & Evaluation

- Training three denoising models for comparison
 - Φ_{b100} : obtained through regular training for 100 epochs
 - Φ_{ut} : obtained by fine-tuning Φ_{b100} with unlikelihood training for 100 epochs
 - Φ_{b200} : obtained by continuing regular training on Φ_{b100} for 100 epochs
- Sample 300 graphs for each model and count constraint violations

2026-07-22

Constrained Graph Diffusion

└ Methodology

└ Training & Evaluation

Training & Evaluation

- Training three denoising models for comparison
 - Φ_{b100} : obtained through regular training for 100 epochs
 - Φ_{ut} : obtained by fine-tuning Φ_{b100} with unlikelihood training for 100 epochs
 - Φ_{b200} : obtained by continuing regular training on Φ_{b100} for 100 epochs
- Sample 300 graphs for each model and count constraint violations

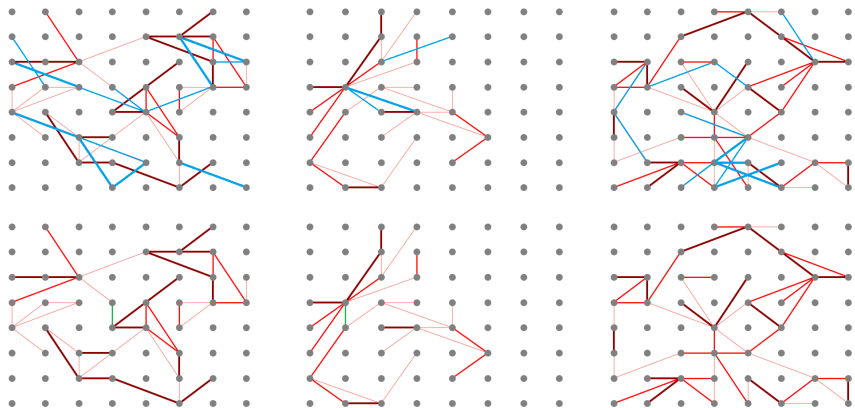
With these methods, we trained three different denoising models: **first one is trained for 100 epochs** and **acts as** a baseline for how well SparseDiff adheres to constraints implicitly imposed by dataset.

Then, we finetune this baseline model for 100 more epochs via unlikelihood training **in order to reduce** the number of constraint violations.

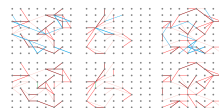
To ensure a fair comparison, we also train a third model, also based on the baseline model, which instead of unlikelihood training continues regular training for 100 epochs.

We then sampled 300 graphs for each model and counted the number

Generated Samples



Graphs sampled from the baseline model Φ_{b100} and their projected versions.



Here you can see three graphs which we generated with the baseline model

The first row contains the raw outputs, while for the **second row we corrected them by applying** a single projection step to the final graphs.

Overall, the baseline quickly learned to generate similar looking graphs that mostly adhere to all the constraints.

Constraint Violations

- Unlikelihood training reduced edge crossings
- Still less effective than continuing regular training
- Longer training led to slightly more disconnected components and illegal edges

Model	Disconn. Components	Edge Crossings		Illegal Edge Lengths	
	Avg. Count ↓	Avg. Count ↓	Rate (%) ↓	Avg. Count ↓	Rate (%) ↓
Φ_{b100}	0.303	15.627	13.537	0.003	0.003
Φ_{b200}	0.493	8.350	8.906	0.020	0.020
Φ_{ut}	0.447	10.853	9.892	0.007	0.006

Constraint violation metrics averaged over 300 generated samples, comparing the baseline model at 100 epochs (Φ_{b100}), 200 epochs (Φ_{b200}), and the model fine-tuned via unlikelihood training (Φ_{ut})

Constrained Graph Diffusion

Results

Constraint Violations

Constraint Violations

- Unlikelihood training reduced edge crossings
- Still less effective than continuing regular training
- Longer training led to slightly more disconnected components and illegal edges

Model	Disconn. Components		Edge Crossings		Illegal Edge Lengths	
	Avg. Count ↓	Rate (%) ↓	Avg. Count ↓	Rate (%) ↓	Avg. Count ↓	Rate (%) ↓
Φ_{b100}	0.303	13.537	8.350	8.906	0.003	0.003
Φ_{b200}	0.493	15.627	10.853	9.892	0.020	0.020
Φ_{ut}	0.447	15.627	10.853	9.892	0.007	0.006

Constraint violation metrics averaged over 300 generated samples, comparing the baseline model at 100 epochs (Φ_{b100}), 200 epochs (Φ_{b200}), and the model fine-tuned via unlikelihood training (Φ_{ut})

In this table you can see avg num of violations per graph. For the **baseline** model, edges with illegal lengths occurred **practically never** and **disconnected graphs were rare with 0.3** disconnected components per graph.

Planarity on the other hand turned out to be way more difficult with **15.6**.

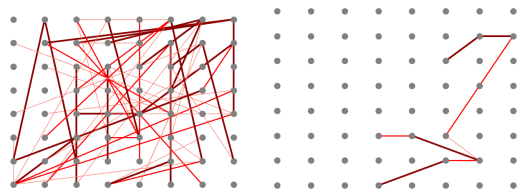
Fine-tuning via unlikelihood training did in fact **reduce down to 10.8**.

However, continuing with regular training turned out to be more effective **8.3**.

Interestingly, continuing with either regular or unlikelihood training increased number of disconnected and illegal **slightly compared** to the baseline.

Projected Diffusion

- Projected intermediate outputs often looked substantially different
- Out-of-distribution samples may confuse the model
 - especially during the first few diffusion steps



(a) an intermediate output before projection

(b) the corrected intermediate graph after projection

2026-07-22

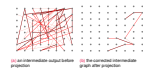
Constrained Graph Diffusion

Results

Projected Diffusion

Projected Diffusion

- Projected intermediate outputs often looked substantially different
- Out-of-distribution samples may confuse the model
 - especially during the first few diffusion steps



Regarding projected diffusion, projecting just the final output **by definition always leads to graphs** that satisfy all constraints.

However, projecting intermediate outputs of every diffusion step often produced graphs that looked **completely different**. For example this one.

These out-of-distribution samples would sometimes **confuse denoising model during subsequent** steps since the model **was trained on completely different graphs** and

this **would sometimes cause the model to produce non-sensical outputs**.

Conclusion

- Promising baseline performance for local constraints
- Global constraints remain more challenging
- Unlikelihood training turned out less effective than continuing regular training
- Projection of final output produces strictly better output at a small cost
- Projection of intermediate outputs may cause problematic distribution shift

Constrained Graph Diffusion

└ Discussion

└ Conclusion

Conclusion

- Promising baseline performance for local constraints
- Global constraints remain more challenging
- Unlikelihood training turned out less effective than continuing regular training
- Projection of final output produces strictly better output at a small cost
- Projection of intermediate outputs may cause problematic distribution shift

To sum everything up

baseline performance looks quite promising for local, however global such as planarity **remain challenging**

Surprisingly, unlikelihood training turned out to be less effective than continuing regular training and is **likely not worth using** because of its overhead. **For PDM, projecting** only the final output **always produces strictly better output** in terms of constraint violations **but**

The projection of intermediate on other hand may cause problematic distribution shifts, **while also slowing down** generation.

Limitations & Future Work

- Evaluate the similarity to true data distribution via well-established metrics
- Explore different unlikelihood training approaches
- Explore different projection strategies
 - periodic intervals
 - time-step dependent probability
 - not always fix all constraints
(e.g., connectivity only during last k steps)
- Evaluate constraint satisfaction behavior for different constraints, domains and datasets

Constrained Graph Diffusion

└ Discussion

└ Limitations & Future Work

Limitations & Future Work

- Evaluate the similarity to true data distribution via well-established metrics
- Explore different unlikelihood training approaches
- Explore different projection strategies
 - periodic intervals
 - time-step dependent probability
 - not always fix all constraints
(e.g., connectivity only during last k steps)
- Evaluate constraint satisfaction behavior for different constraints, domains and datasets

Clearly, this project is not without its limitations.

For example, we focused solely on validity and did not evaluate similarity to true data distribution

It has to be also kept in mind, that we tried **only one concrete implementation** of projected diffusion and unlikelihood training. **Different approaches might lead to different results.**

For example, instead of projecting at every time step one could do so with a certain probability depending on current time step or to not always fix all constraints.

Constraint satisfaction may also look quite different for other domains, constraints or **real word datasets**, which are usually less structured than our toy dataset.

Questions?

Do you have any questions?

└ SparseDiff - Noise Model

SparseDiff - Noise Model

$$q(G^t|G) = (\mathbf{X} \bar{\mathbf{Q}}_X^t, \mathbf{Y} \bar{\mathbf{Q}}_Y^t) \quad \bar{\mathbf{Q}}_X^t = \prod_{i=1}^t \mathbf{Q}_X^i = \alpha^t \mathbf{I} + (1 - \alpha^t) \mathbf{1}_a \mathbf{p}_a^\top \quad \mathbf{Q}_X^t = \alpha^t \mathbf{I} + (1 - \alpha^t) \mathbf{1}_a \mathbf{p}_a^\top$$

$$\bar{\mathbf{Q}}_Y^t = \prod_{i=1}^t \mathbf{Q}_Y^i = \alpha^t \mathbf{I} + (1 - \alpha^t) \mathbf{1}_b \mathbf{p}_b^\top \quad \mathbf{Q}_Y^t = \alpha^t \mathbf{I} + (1 - \alpha^t) \mathbf{1}_b \mathbf{p}_b^\top$$

Equations for the direct computation of the noisy graph distribution

1. **Sampling new edge count:** Let $\bar{m}_t = \binom{n}{2} - m_t$ denote the number of non-existing edges in G^t . The number of new edges is sampled as $k \sim \mathcal{B}(\bar{m}_t, q_t)$, where $q_t = 1 - \mathbf{Q}^t[0, 0]$ represents the probability of a non-existing edge transitioning to an existing one.
2. **Sampling positions:** The new edge positions are selected uniformly at random from E_{ne} .
3. **Sampling edge attributes:** The attribute of each new edge is drawn from the Multinomial distribution $\text{Multinomial}(\mathbf{Q}^t[0, 1 : b])$, representing the transition probabilities over existing edge types.

Qin, Vignac, and Frossard, "SparseDiff: Sparse Discrete Diffusion for Scalable Graph Generation"

SparseDiff - Denoising Network

Algorithm 1 Sparse training at step t with the sparsity parameter λ (Section 3.1 & 3.2)

- 1: Given the clean graph $G = (E^0, X^0, Y^0)$;
- 2: Sample the noisy graph $G^t = (E^t, X^t, Y^t)$;
- 3: Sample query edges E_q of size $\lceil \lambda n^2 \rceil$;
- 4: $E_m \leftarrow E^t \cup E_q, Y_m \leftarrow Y^t \cup Y_q$ ▷ Construct message-passing edges
- 5: $G_m \leftarrow (E_m, X^t, Y_m)$ ▷ Construct the message-passing graph
- 6: $(\hat{P}^X, \hat{P}^Y) = \phi_\theta(G_m, E_q)$ ▷ Predict the distribution of nodes and query edges
- 7: optimizer.step($\text{CE}(\hat{X}^0, \hat{P}^X) + \text{CE}(Y_q^0, \hat{P}^Y)$) ▷ Loss calculation

Qin, Vignac, and Frossard, "SparseDiff: Sparse Discrete Diffusion for Scalable Graph Generation"

└ SparseDiff - Denoising Network

SparseDiff - Iterative Interference

Algorithm 2 Iterative inference at step t with the sparsity parameter λ (Section 3.3)

```

1: Initialize an empty graph  $G^{t-1}$  with unlabeled nodes  $\mathbf{X}^{t-1}$  and no edges;
2: Randomly divide all node pairs into  $K = \lceil \frac{1}{\lambda} \rceil$  equal-sized chunks  $\{C_0, \dots, C_{K-1}\}$ ;
3: for  $k$  in  $\text{range}(K)$  do
4:    $\mathbf{E}_q \leftarrow C_k$ ; ▷ Set query edges
5:    $\mathbf{E}_m \leftarrow \mathbf{E}^t \cup \mathbf{E}_q$ ; ▷ Construct message-passing edges and its attributes  $\mathbf{Y}_m$ 
6:    $G_m \leftarrow (\mathbf{E}_m, \mathbf{X}^t, \mathbf{Y}_m)$ ; ▷ Construct the message-passing graph
7:    $(\hat{\mathbf{P}}^{\mathbf{X}}, \hat{\mathbf{P}}^{\mathbf{Y}}) = \phi_\theta(G_m, \mathbf{E}_q)$ ; ▷ Predict the distribution of nodes and query edges
8:    $\hat{\mathbf{X}} = \text{Multinomial}(\hat{\mathbf{P}}^{\mathbf{X}})$ ,  $\hat{\mathbf{Y}}_q = \text{Multinomial}(\hat{\mathbf{P}}^{\mathbf{Y}})$ ; ▷ Sample labels
9:    $\mathbf{X}^{t-1} \leftarrow \hat{\mathbf{X}}$ ; ▷ Assign node new labels
10:   $\mathbf{Y}^{t-1} \leftarrow \mathbf{Y}^{t-1} \cup \hat{\mathbf{Y}}_q[\hat{\mathbf{Y}}_q! = 0]$ ,  $\mathbf{E}^{t-1} \leftarrow \mathbf{E}^{t-1} \cup \hat{\mathbf{E}}_q[\hat{\mathbf{Y}}_q! = 0]$ ; ▷ Add existing edges

```

Qin, Vignac, and Frossard, “SparseDiff: Sparse Discrete Diffusion for Scalable Graph Generation”

└ SparseDiff - Iterative Interference

Algorithm 2 Iterative inference at step t with the sparsity parameter λ (Section 3.3)

```

1: Initialize an empty graph  $G^{t-1}$  with unlabeled nodes  $\mathbf{X}^{t-1}$  and no edges;
2: Randomly divide all node pairs into  $K = \lceil \frac{1}{\lambda} \rceil$  equal-sized chunks  $\{C_0, \dots, C_{K-1}\}$ ;
3: for  $k$  in  $\text{range}(K)$  do
4:    $\mathbf{E}_q \leftarrow C_k$ ; ▷ Set query edges
5:    $\mathbf{E}_m \leftarrow \mathbf{E}^t \cup \mathbf{E}_q$ ; ▷ Construct message-passing edges and its attributes  $\mathbf{Y}_m$ 
6:    $G_m \leftarrow (\mathbf{E}_m, \mathbf{X}^t, \mathbf{Y}_m)$ ; ▷ Construct the message-passing graph
7:    $(\hat{\mathbf{P}}^{\mathbf{X}}, \hat{\mathbf{P}}^{\mathbf{Y}}) = \phi_\theta(G_m, \mathbf{E}_q)$ ; ▷ Predict the distribution of nodes and query edges
8:    $\hat{\mathbf{X}} = \text{Multinomial}(\hat{\mathbf{P}}^{\mathbf{X}})$ ,  $\hat{\mathbf{Y}}_q = \text{Multinomial}(\hat{\mathbf{P}}^{\mathbf{Y}})$ ; ▷ Sample labels
9:    $\mathbf{X}^{t-1} \leftarrow \hat{\mathbf{X}}$ ; ▷ Assign node new labels
10:   $\mathbf{Y}^{t-1} \leftarrow \mathbf{Y}^{t-1} \cup \hat{\mathbf{Y}}_q[\hat{\mathbf{Y}}_q! = 0]$ ,  $\mathbf{E}^{t-1} \leftarrow \mathbf{E}^{t-1} \cup \hat{\mathbf{E}}_q[\hat{\mathbf{Y}}_q! = 0]$ ; ▷ Add existing edges

```

Qin, Vignac, and Frossard, “SparseDiff: Sparse Discrete Diffusion for Scalable Graph Generation”

Projected Diffusion

Algorithm 1: PDM

```
1  $x_T^0 \sim \mathcal{N}(\mathbf{0}, \sigma_T \mathbf{I})$ 
2 for  $t = T$  to 1 do
3    $\gamma_t \leftarrow \sigma_t^2 / 2\sigma_T^2$ 
4   for  $i = 1$  to  $M$  do
5      $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I});$ 
6      $\mathbf{g} \leftarrow s_{\theta^*}(x_t^{i-1}, t)$ 
7      $x_t^i = \mathcal{P}_C(x_t^{i-1} + \gamma_t \mathbf{g} + \sqrt{2\gamma_t} \epsilon)$ 
8    $x_{t-1}^0 \leftarrow x_t^M$ 
9 return  $x_0^0$ 
```

Christopher, Baek, and Fioretto, “Constrained Synthesis with Projected Diffusion Models”

└ Projected Diffusion

Projected Diffusion

Algorithm 1: PDM

```
1  $x_T^0 \sim \mathcal{N}(\mathbf{0}, \sigma_T \mathbf{I})$ 
2 for  $t = T$  to 1 do
3    $\gamma_t \leftarrow \sigma_t^2 / 2\sigma_T^2$ 
4   for  $i = 1$  to  $M$  do
5      $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I});$ 
6      $\mathbf{g} \leftarrow s_{\theta^*}(x_t^{i-1}, t)$ 
7      $x_t^i = \mathcal{P}_C(x_t^{i-1} + \gamma_t \mathbf{g} + \sqrt{2\gamma_t} \epsilon)$ 
8    $x_{t-1}^0 \leftarrow x_t^M$ 
9 return  $x_0^0$ 
```

Christopher, Baek, and Fioretto, "Constrained Synthesis with Projected Diffusion Models"